Game design document — asteroids

Authors

Tim van Naarden
t.vannaarden@students.uu.nl

Matt ter Steege
m.j.tersteege@students.uu.nl

October 30, 2026

Contents

1 Game concept . . . **2**
1.1 Game type and objective . . . 2
2 Core gameplay . . . **2**
2.1 Controls . . . 2
2.2 Rules and progression . . . 2
3 Design . . . **2**
3.1 Data structures . . . 2
3.2 Computer adversary . . . 5
3.3 Interface . . . 6
4 Minimal requirements . . . **6**
4.1 Keyboard-controlled player . . . 6
4.2 Enemies with basic intelligence . . . 6
4.3 Randomness . . . 6
4.4 Multi-frame animation . . . 7
4.5 Pause . . . 7
4.6 File-system interaction . . . 7
5 Optional requirements . . . **7**
5.1 Automated testing . . . 7
5.2 Saving the game state as JSON . . . 8
6 Pure vs impure code . . . **8**
7 Abstractions . . . **9**
7.1 Type classes . . . 9
7.2 Higher-order functions . . . 9

1 | Game concept

Game type and objective

The game is a two-dimensional arcade game inspired by *Asteroids*. The player pilots a spaceship in an arena that wraps around at the screen edges. Asteroids drift through the arena; the player rotates, accelerates, and fires projectiles to destroy them while avoiding collisions. As the score increases, the game introduces more pressure through additional asteroids and enemy ships with simple pursuit behaviour.

The main objective is to survive for as long as possible and achieve the highest score. The game ends when the player runs out of lives. The visual style will use geometric shapes rendered with Gloss rather than external artwork.

In addition to standard drifting asteroids, hostile scout ships will appear after the player has survived for a while. Scouts steer towards the player's current position, but have a limited turning rate and speed so that their behaviour is predictable and testable. Larger asteroids split into smaller asteroids when destroyed.

2 | Core gameplay

Controls

Input	Action
Left / Right arrows or a / d keys	Rotate the ship
Up arrow or w	Apply thrust in the facing direction
Space	Fire a projectile
P	Pause or resume the game
Enter	Start a new game or restart after game over
Escape	Return to the title screen or quit from the title screen
F5	Save the current game
F9	Load the saved game

Rules and progression

The ship has a limited number of lives (3). A collision with an asteroid or enemy costs one life and briefly makes the ship invulnerable, preventing removing several lives at once. When an asteroid is hit by a bullet shot by the player, it is removed and awards points. Large and medium asteroids split into smaller ones; the smallest asteroids disappear when destroyed. Enemy ships award more points than asteroids.

The arena uses wrap-around boundaries: an object leaving one edge re-enters on the opposite edge. New asteroids and enemy ships appear at random positions, preferably away from the player to avoid unavoidable collisions. The spawn rate and the number of active threats increase gradually with elapsed play time or score. A game-over screen displays the final score and the locally stored high score.

3 | Design

We follow the model-view-controller structure of the Gloss template. This means that all data types are defined in the `Model` module, the drawing happens in `View`, and the updates happen in `Controller`. The listings in this section are taken directly from our code. `Point` and `Vector` are the Gloss types for a position and a direction, which are both `(Float, Float)`.

Data structures

The game state

The full state of our game is represented using a `GameState` data type:

```
data GameState = GameState {
    gameStatus :: GameStatus,     -- Which screen is shown and whether the game is running
    playerData :: PlayerData,     -- The current player
```

```
    asteroids :: [Asteroid],      -- Asteroids currently in the arena
    highScore :: Int,             -- Best score so far, read from file at startup
    score :: Int,                 -- Score of the current game
    elapsedTime :: Float,         -- Seconds since the game started
    enemies :: [EnemyData],       -- Enemies that are currently alive
    bullets :: [Bullet],          -- Bullets that are currently flying
    explosions :: [Explosion],    -- Explosions that are currently animating
    randomSeed :: Seed            -- Passed through every function that needs randomness
} deriving (Show, Eq)
```

The state contains exactly one `PlayerData`, so the type system already guarantees that there is always a single ship. Asteroids, enemies, bullets, and explosions each have their own list with their own type, instead of one generic list of game objects. As a result, a function that updates asteroids can never receive an enemy by accident. `elapsedTime` is used as the clock of the game. Timers, such as the invulnerability after a death, store the moment they started, which we then compare with `elapsedTime`.

The player data

A player is represented as follows:

```
-- | Number of lives the player starts a new game with
startingLives :: Int
startingLives = 3

-- | Seconds the player stays invincible after spawning or dying
invincibilityDuration :: Float
invincibilityDuration = 2

-- | Minimum number of seconds between two shots of the player
fireCooldown :: Float
fireCooldown = 0.25

data PlayerData = PlayerData {
  playerLocation :: Point,
  livesRemaining :: Int,     -- Starts at startingLives
  playerRotation :: Float,   -- Between 0 and 360
  playerVelocity :: Vector, -- Horizontal and vertical speed, thrust adds speed in the
    playerRotation direction
  playerLastDeath :: Float, -- Used for invincibility timer, also grants invincibility on spawn
  playerLastShot :: Float    -- Used for the firing cooldown
} deriving (Show, Eq)
```

We store the rotation and the velocity separately. Thrust adds speed in the direction the ship is facing, but when the ship turns, it keeps drifting in its old direction, just like in the original game. `playerLastDeath` holds the value of `elapsedTime` at the moment of the last death. The player is invulnerable as long as less than `invincibilityDuration` seconds have passed since then. By setting this field to the current time at the start of a game, the player is also protected right after spawning.

`playerLastShot` works in the same way for shooting. A new bullet is only fired when at least `fireCooldown` seconds have passed since the last shot. Without this cooldown, holding the space bar would fire a bullet every frame, which would make the game far too easy.

The enemies

Enemies are represented using the following data type:

```
data EnemyData = EnemyData {
  enemyLocation :: Point,
  enemyRotation :: Float, -- Between 0 and 360
  enemyVelocity :: Vector -- Horizontal and vertical speed, the enemy steers it towards the player
} deriving (Show, Eq)
```

They can shoot bullets at the player's current location. How they decide in which direction to move is described in the computer adversary section (Section 3.2).

Bullets

Bullets can be fired either by the player or by one of the enemies:

```
data BulletType = PlayerBullet | EnemyBullet deriving (Show, Eq)

data Bullet = Bullet {
  bulletLocation :: Point,
  bulletVelocity :: Vector, -- Bullets have no rotation (just a dot), so this is the horizontal and
    vertical speed
  bulletType :: BulletType
} deriving (Show, Eq)
```

All bullets are stored in one list, and `bulletType` determines what a bullet is able to hit. A `PlayerBullet` destroys asteroids and enemies, while an `EnemyBullet` can only hit the player. This prevents enemies from destroying each other or the asteroids.

The asteroids

When an asteroid is hit, it splits into two smaller asteroids. Only the smallest asteroids disappear completely.

```
data AsteroidSize = Small | Medium | Large deriving (Enum, Show, Eq, Bounded)

data Asteroid = Asteroid {
  asteroidLocation :: Point,
  asteroidSize :: AsteroidSize,
  asteroidVelocity :: Vector -- Asteroids have no rotation, so this is the horizontal and vertical
    speed
} deriving (Show, Eq)
```

Because `AsteroidSize` derives `Enum` and `Bounded`, we do not need a separate case for every size when splitting. A hit asteroid is replaced by two asteroids of the previous size (using `pred`), and an asteroid of size `minBound` is removed. The size also determines the radius that is used for drawing and collision checks, and the number of points the player receives.

```
splitAsteroid  :: Asteroid → [Asteroid]
asteroidRadius :: AsteroidSize → Float
asteroidPoints :: AsteroidSize → Int
```

Explosions

Whenever an asteroid, an enemy, or the player is destroyed, an explosion is added to the game state:

```
data Explosion = Explosion {
  explosionLocation :: Point,
  explosionAge :: Float,      -- Seconds since the explosion started
  explosionDuration :: Float -- The explosion is removed once its age reaches this
} deriving (Show, Eq)
```

In contrast to the timers of the player, an explosion stores its own age instead of a start time. The reason for this is that the drawing of an explosion depends on how far along it is. For example, the radius of the burst can be computed directly from `explosionAge / explosionDuration`. Since every explosion has its own duration, the explosion of the player can last longer than that of a small asteroid.

Game status

The game has several screens, and the game should only update while it is actually being played. Therefore, the current screen is stored in the game state as a `GameStatus`:

```haskell
data GameStatus = Title | Playing | Paused | GameOver deriving (Show, Eq)
```

Both the update and the input functions first look at `gameStatus`. For example, Enter only starts a new game on the `Title` and `GameOver` screens, and P only has an effect while the status is `Playing` or `Paused`.

Random generator

In order to keep randomness out of IO, we store the state of our random generator in the game state:

```haskell
-- | State of the random generator, a Word32 overflows at 2^32 so the modulo comes for free
newtype Seed = Seed Word32 deriving (Show, Eq)
```

We chose a `newtype` so that a seed cannot accidentally be mixed up with other numbers in the game, such as the score. The generator itself is described in the randomness section (Section 4.3). A `Word32` is an unsigned 32-bit number, which means that it overflows at 2^{32}. As a result, the modulo in the formula of the generator comes for free.

Moving objects

The player, enemies, asteroids, and bullets all have a location and a velocity, and they all move in the same way every update. Instead of writing a separate move function for each type, we let them share the type class `Movable`:

```haskell
-- | Everything that drifts through the arena with a velocity
class Movable a where
  location    :: a → Point
  velocity    :: a → Vector
  setLocation :: Point → a → a

-- | Move an object along its velocity for the given number of seconds
move :: Movable a ⇒ Float → a → a
move secs x = setLocation (px + vx * secs, py + vy * secs) x
  where (px, py) = location x
        (vx, vy) = velocity x

instance Movable Asteroid where
  location        = asteroidLocation
  velocity        = asteroidVelocity
  setLocation p x = x { asteroidLocation = p }
```

The instances for `PlayerData`, `EnemyData`, and `Bullet` follow the same pattern. This means that `move` is only defined once, and updating all asteroids becomes `map (move secs) (asteroids gstate)`. Wrapping around the screen edges and checking for collisions only depend on the location as well. Therefore, these functions will also be written once for every `Movable`:

```haskell
wrapAround :: Movable a ⇒ a → a
collides   :: (Movable a, Movable b) ⇒ Float → a → b → Bool
```

Computer adversary

After the player has survived for a while, scout ships will start to appear. Each update, a scout computes the angle from its own location to the player and turns `enemyRotation` towards it. However, it can never turn faster than a fixed turning rate. Its velocity is then accelerated in the new direction, up to a maximum speed. As a result, scouts chase the player, but they are never impossible to escape. When the player is roughly in front of a scout, it fires an `EnemyBullet`.

```haskell
updateEnemy :: Float → PlayerData → EnemyData → EnemyData
enemyShoot  :: PlayerData → EnemyData → Maybe Bullet
```

Both functions are pure and only depend on the player and the enemy itself. This makes it possible to test the steering without running the game.

Interface

The game is drawn with Gloss using `playIO`. Gloss requires the view function to work in IO, even though drawing does not need any side effects. Therefore, the view function only calls a pure function that builds the picture:

```
view     :: GameState → IO Picture
viewPure :: GameState → Picture
```

`viewPure` combines a picture for every element of the game state using `pictures`. The ship is drawn as a triangle rotated by `playerRotation`, asteroids as circles with a radius based on their size, and bullets as small dots. Each of these types will be an instance of the `Drawable` class, which is described in the abstractions section (Section 7). On top of the arena, the score, the high score, and the remaining lives are shown as text. The `Title`, `Paused`, and `GameOver` statuses each get their own overlay.

```
drawHud    :: GameState → Picture
drawScreen :: GameStatus → Picture
```

Keyboard input is handled in the same way. The IO version required by `playIO` calls a pure function that translates a key event into a change of the game state, following the controls in the table above.

```
input    :: Event → GameState → IO GameState
inputKey :: Event → GameState → GameState
```

4 | Minimal requirements

Keyboard-controlled player

The `PlayerData` type stores position, velocity, orientation, the time of the last shot for the firing cooldown, and the time of the last death for temporary invulnerability. `inputKey` in the `Controller` module maps keyboard events to rotation, thrust, firing, pause, and restart actions. Movement uses the "delta" time supplied by Gloss so that motion is not tied to a particular frame rate.

Enemies with basic intelligence

Enemy scouts use a pursuit rule. Each update, an enemy estimates the direction from its position to the player, turns towards that direction at a bounded rate, and accelerates or fires when the player is suitably aligned. The "AI" logic will be isolated in a separate `AI` module, making it possible to test steering decisions independently of rendering.

Randomness

The initial asteroid field, spawn positions, asteroid velocities, and timing of new enemies will include random variation. Randomness will be handled through an explicit random generator rather than hidden global state, which is stored as the `randomSeed` field of the game state (Section 3.1.8). Spawn functions will receive the generator and return both the generated entities and the updated generator state. Spawn positions will be checked against a minimum distance from the player where practical.

The used function for the random number generator will be the one used in the random function in C:

$$X_{n+1} = (aX_n + c) \bmod 2^{32} \quad \text{where} \quad a = 1103515245 \quad \text{and} \quad c = 12345$$

```
nextSeed    :: Seed → Seed
randomRange :: (Float, Float) → Seed → (Float, Seed)
spawnAsteroid :: PlayerData → Seed → (Asteroid, Seed)
```

Multi-frame animation

Explosions will be represented as timed effects with a position, age, and duration, using the `Explosion` type (Section 3.1.6). Each update advances their age; the renderer draws a short-lived expanding burst or a collection of particles. Effects expire after their duration. This gives animation a clear state representation and keeps drawing separate from state transitions.

Pause

The `GameStatus` type (Section 3.1.7) represents the title, playing, paused, and game-over states. Pressing P switches between `Playing` and `Paused`. While paused, the update function will not advance positions, cooldowns, spawn timers, or animation ages. Input needed to resume or return to the title screen will still be processed.

File-system interaction

The game will load a high-score value at startup and write a new value when the player beats it. The `Persistence` module will isolate file operations and handle a missing or invalid file by falling back to a default score. The initial format can be a simple text file, which satisfies the requirement without adding a serialization dependency unrelated to the core game.

5 | Optional requirements

Automated testing

Several parts of the game have properties that should hold for every possible input. Therefore, we will test these parts with QuickCheck. The tests will be placed in a separate test suite in the Cabal file, so that they do not become part of the game itself. In order to generate random game objects, the types from the `Model` module will get `Arbitrary` instances. For `AsteroidSize`, the derived `Enum` and `Bounded` instances already give us every possible value:

```haskell
instance Arbitrary AsteroidSize where
  arbitrary = elements [minBound .. maxBound]

instance Arbitrary Asteroid where
  arbitrary = Asteroid <$> arbitrary <*> arbitrary <*> arbitrary
```

The properties will cover movement, asteroid splitting, the steering of enemies, and saving the game state (Section 5.2):

```haskell
-- Moving for zero seconds does not change anything
prop_moveZero :: Asteroid → Bool
prop_moveZero a = move 0 a == a

-- After wrapping, an object is always inside the arena
prop_wrapInside :: Asteroid → Bool
prop_wrapInside a = insideArena (location (wrapAround a))

-- Splitting only produces smaller asteroids
prop_splitSmaller :: Asteroid → Bool
prop_splitSmaller a = all smaller (splitAsteroid a)
  where smaller b = fromEnum (asteroidSize b) < fromEnum (asteroidSize a)

-- An enemy never turns faster than its maximum turning rate
prop_turnRate :: Positive Float → PlayerData → EnemyData → Bool

-- Saving and loading a game state gives back the same state
prop_jsonRoundTrip :: GameState → Bool
prop_jsonRoundTrip gs = decode (encode gs) == Just gs
```

These properties are only possible because the functions involved are pure. They take a value and return a new one, without depending on the screen, the keyboard, or the file system.

Saving the game state as JSON

Pressing F5 will save the full game state to a file, and pressing F9 will load it again. We will store the state as JSON using the Aeson library. Instead of writing the conversion by hand, every type in the game state will derive `Generic`. After that, Aeson can generate the `ToJSON` and `FromJSON` instances for us:

```
{-# LANGUAGE DeriveGeneric #-}

data GameState = GameState { ... } deriving (Show, Eq, Generic)

instance ToJSON GameState
instance FromJSON GameState
```

The same is done for `PlayerData`, `EnemyData`, `Asteroid`, `AsteroidSize`, `Bullet`, `BulletType`, `Explosion`, `GameStatus`, and `Seed`. `Point` and `Vector` are tuples of floats, which Aeson already supports. Since a `Seed` is just a single number, it is saved as well. This means that a loaded game continues with exactly the same random sequence.

Reading and writing the file will be handled by two small functions in the `Persistence` module:

```
saveGame :: FilePath → GameState → IO ()
saveGame = encodeFile

loadGame :: FilePath → IO (Maybe GameState)
loadGame = decodeFileStrict
```

When the file is missing or cannot be decoded, `loadGame` returns `Nothing` and the current game simply continues. A loaded game starts with the status `Paused`, so that the player has a moment to see the situation before playing on. The high score remains a separate plain text file, so it is not lost when a save file is overwritten.

6 | Pure vs impure code

Gloss expects the step, input, and view functions to work in IO. However, most of the game does not need any side effects. Therefore, each of these functions will consist of a thin IO layer around a pure function that contains the actual logic:

```
step     :: Float → GameState → IO GameState
stepPure :: Float → GameState → GameState

input    :: Event → GameState → IO GameState
inputKey :: Event → GameState → GameState

view     :: GameState → IO Picture
viewPure :: GameState → Picture
```

The impure parts of the game are limited to the following:

1. Reading the high score at startup in `main`, and writing it in `step` when a game ends with a new high score.
2. Saving and loading the game state in `input` when F5 or F9 is pressed.
3. Choosing the first `Seed` once at startup, for example based on the current time.

All file operations will be placed in the `Persistence` module, so the other modules do not need to import anything related to IO. Because the random generator is stored explicitly in the game state, spawning asteroids and enemies is pure as well. A spawn function receives a `Seed` and returns the new objects together with the next `Seed`. Movement, collisions, splitting asteroids, the steering of enemies, animations, and drawing are all pure functions of the game state. As a result, they can all be tested with QuickCheck, as described in the automated testing section.

7 | Abstractions

Type classes

The `Movable` class (Section 3.1.9) removes the duplication between the four moving types. This means that `move`, `wrapAround`, and `collides` are each written once instead of four times. We will handle drawing in the same way with a `Drawable` class:

```
class Drawable a where
  draw :: a → Picture

viewPure :: GameState → Picture
viewPure gs = pictures (  [draw (playerData gs)]
                       ++ map draw (asteroids gs)
                       ++ map draw (enemies gs)
                       ++ map draw (bullets gs)
                       ++ map draw (explosions gs)
                       ++ [drawHud gs, drawScreen (gameStatus gs)] )
```

`PlayerData`, `EnemyData`, `Asteroid`, `Bullet`, and `Explosion` will each have a `Drawable` instance. An explosion is not `Movable`, since it stays at the same location, but it can still be drawn. This is also why we chose two separate classes instead of one big class for all game objects. The JSON instances (`ToJSON` and `FromJSON`) and the test instances (`Arbitrary`) are type classes as well. The first two are derived generically, so they do not require any duplicated code.

Higher-order functions

Since every kind of object is stored in a list, most updates can be written with higher-order functions instead of explicit recursion. For example:

1. Moving all objects is a `map` over each list.
2. Destroyed objects and finished explosions are removed with `filter`.
3. Hit asteroids are replaced by their pieces using `concatMap splitAsteroid`.
4. The score for all destroyed asteroids is computed with a fold over their sizes.

In addition, constants such as `startingLives`, `invincibilityDuration`, and `fireCooldown` are defined once with a name. Therefore, no magic numbers appear in the update functions.